



Octave code for the wrapper (1)

- In this lesson, you will learn how to implement the EKF parameter estimator in Octave, and will see results of applying it to the linear spring-mass-damper model.
 - Note that even though the model is linear, we still require a nonlinear KF to perform parameter estimation.
 - The unknown parameters are: $\theta = [k, b, m]^T$.
- The Octave wrapper code loads data and reserves storage for results.

```
% Load data file comprising model parameters and simulation profile
load simOut.mat model dT t u x z

% Reserve storage for computed results, for plotting
[nx,nt] = size(x);
nq      = 3; % this many parameters to estimate
qhatstore = zeros(nq,nt);
boundstore = zeros(nq,nt);
```



Octave code for the wrapper (2)

- The wrapper code then defines true parameters q , defines our initial guess of θ , its covariance, and some noise matrices. Then, it initializes the EKF data structures.

```
% Covariance values
q      = [model.k, model.b, model.m]';
q0     = [2;1;40];
SigmaQ0 = diag((q-q0)/3)^2; % uncertainty of initial parameters
SigmaW  = diag(1e-8*[1 1 10]);
SigmaE  = blkdiag(model.SigmaW,model.SigmaV);

% Create ekfData structure and initialize variables
ekfData = initEKFid(q0,SigmaQ0,SigmaW,SigmaE,u(1),x(:,1),dT);
```



Octave code for the wrapper (3)

- The main loop is very similar to the one used in a EKF for state estimation.

```
% Now, enter loop for remainder of time, where we update the EKF
% once per sample interval
for k = 2:length(t)
    uk = u(k); % "measure" input force
    xk = x(:,k); % "measure" true state
    zk = z(k); % "measure" nonlinear output

    % Update parameters
    [qhatstore(:,k),boundstore(:,k),ekfData] = iterEKFid(uk,xk,zk,dT,ekfData);
end
```

- Plotting code is also very similar, so I have omitted it from this lesson.



Octave code for initEKFid

- The initEKFid function stores parameters in ekfData.

```
function ekfData = initEKFid(q0,SigmaQ0,SigmaW,SigmaE,u1,x1,dT)
    ekfData.qhat = q0; % Initial state description
    ekfData.SigmaQ = SigmaQ0; % Initial estimation-error covariance
    ekfData.SigmaE = SigmaE; % Sensor-noise covariance (on [xk;zk])
    ekfData.SigmaW = SigmaW; % Process-noise covariance (driving Q)
    ekfData.priorU = u1; % Previous value of input force
    ekfData.priorX = x1; % Previous value of state
    ekfData.Qbump = 5; % In case SigmaX collapses

    % Compute the derivative of the output with respect to the parameters
    syms k b m x1old x2old uold
    sA = [0 1; -k/m -b/m]; sB = [0; 1/m];
    sAd = expm(sA*dT); sBd = (sA\((sAd-eye(2)))*sB;
    sxold = [x1old; x2old];
    sJ = simplify(jacobian(sAd*sxold + sBd*uold,[k b m]));
    ekfData.J = matlabFunction(sJ);
```

- The derivative is a MESS, so I do it symbolically.



Octave code for iterEKFid (1)

- The iteration code in iterEKFid first unpacks parameters from ekfData :

```
function [qhat,bounds,ekfData] = iterEKFid(uk,xk,zk,dT,ekfData)

    % Get data stored in ekfData structure
    priorU = ekfData.priorU;
    priorX = ekfData.priorX;
    SigmaQ = ekfData.SigmaQ;
    SigmaE = ekfData.SigmaE;
    SigmaW = ekfData.SigmaW;
    qhat = ekfData.qhat;
```



Octave code for iterSPKFid (2)

- Then, it executes step 1a to 1c.

```
% EKF Step 1a: Parameter prediction time update
% - Do nothing. qhat(minus,k) = qhat(plus,k-1)

% EKF Step 1b: Prediction-error covariance time update
SigmaQ = SigmaQ + SigmaW;

% Step 1c: Output estimate
k = qhat(1); b = qhat(2); m = qhat(3);
A = [0 1; -k/m -b/m]; % Specify continuous-time model
B = [0; 1/m];
C = [1 0];
Ad = expm(A*dT); Bd = (A\((Ad-eye(2)))*B;
dhat = [Ad*priorX + Bd*priorU; C*xk]; % [xk;zk] pred
```



Octave code for iterSPKFid (3)

- It continues with 2a–2c.

```
% Step 2a: Estimator gain matrix
% Chat = subs(sJ,{sk,sb,sm,sx1old,sx2old,suold}, ...
% {k,b,m,priorX(1),priorX(2),priorU})
Chat = ekfData.J(b,k,m,priorU,priorX(1),priorX(2));
Chat = real([Chat; 0 0 0]); % some operations result in tiny imaginary #s

SigmaD = Chat*SigmaQ*Chat' + SigmaE;
L = SigmaQ*Chat'/SigmaD;

% Step 2b: State estimate measurement update
r = [xk;zk] - dhat; % innovation: could use to check for sensor errors...
qhat = qhat + L*r;

% Step 2c: Error covariance measurement update
SigmaQ = SigmaQ - L*SigmaD*L';
[~,S,V] = svd(SigmaQ); % Implement Higham method to help robustness
HH = V*S*V';
SigmaQ = (SigmaQ + SigmaQ' + HH + HH')/4;
```



Octave code for iterSPKFid (4)

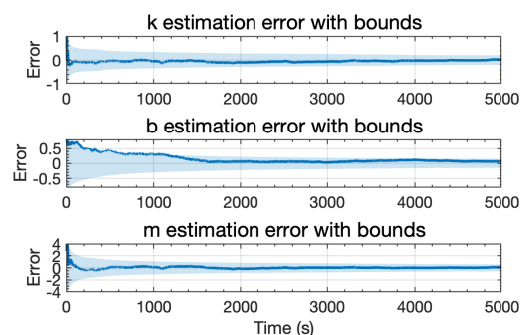
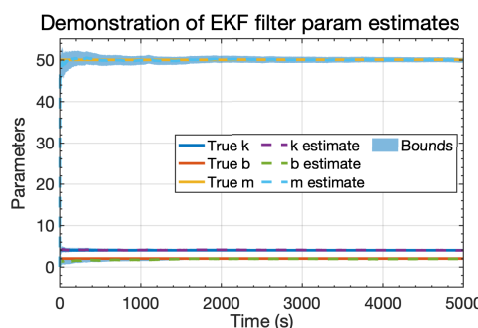
- Finally, it saves results and returns.

```
% Save data in ekfData structure for next time...
ekfData.priorU = uk;
ekfData.priorX = xk;
ekfData.SigmaQ = SigmaQ;
ekfData.qhat = qhat;
bounds = 3*sqrt(diag(SigmaQ));
end
```



Sample results

- Some sample results are shown below: the filter is able to converge to the neighborhood of the k estimate very quickly; m takes a little longer; and b takes the longest.
- Nearly indistinguishable from SPKF-generated results for this example.





Summary

- You have learned how to implement EKF code for parameter estimation, assuming that the state vector is known exactly at all points in time.
- Once again, the code closely follows the steps derived in the previous lesson.
- The biggest challenge with implementing the EKF is finding $\hat{C}_k$.
 - I used symbolic methods because when I write out the derivative it requires three pages to do so!
 - It is very easy to compute the derivative incorrectly if you are doing it manually.
- You have seen that results of parameter estimation, when applied to the linear spring-mass-damper model look very good, and very similar to the SPKF results.